



CSN6214 - Operating Systems

Assignment Report: Advanced Concurrency & IPC Assignment

Instructor: Sharaf Eldeen Sami Mohammed Al Horani

Class Section: TC1L / TT1L

Group 8

Group Members	Student ID
David Ting Zi Xiang	242UC244PE
Lee Kah Wai	243UC247K0
Wong Kai Shen	243UC247DH
Yap Yong Zheng	1211108445

Table Of Contents

1.0 Introduction & Architecture.....	2
2.0 Implementation Details.....	3
2.1 System Calls (File I/O).....	3
2.2 Multi-Threading Implementation.....	3
2.3 Multiprocessing + IP.....	3
2.4 The N-Way Merge Algorithm.....	4
3.0 Experimental Results.....	4
4.0 Analysis & Discussion.....	6
4.1 Speedup vs Number of Workers.....	6
4.2 Reasons for Diminishing Returns.....	6
4.3 Application of Amdahl's Law.....	6
4.4 Comparison of Threads and Processes.....	6
5.0 Conclusion.....	6
6.0 Appendix.....	7
6.1 readme.txt.....	7
6.2 parallel_sort.c.....	7
6.3 Makefile.....	13

1.0 Introduction & Architecture

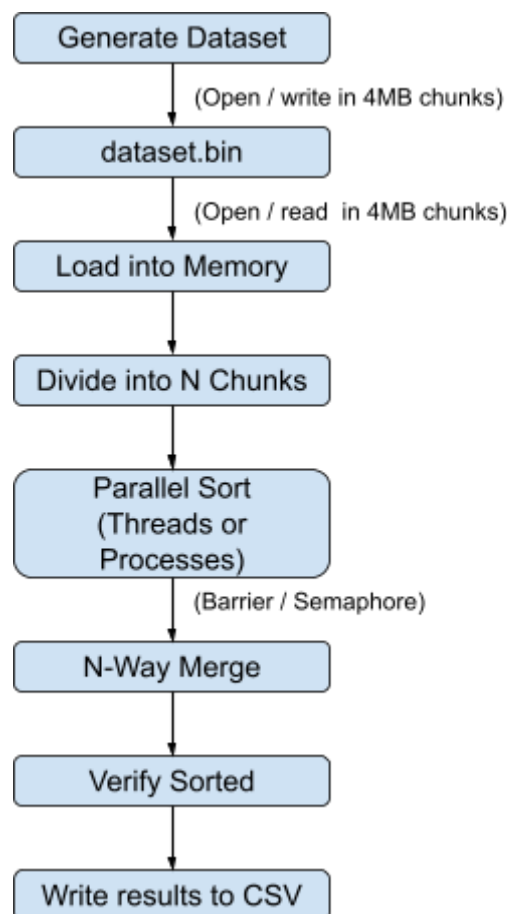
This project implements a parallel merge sort in C using POSIX interface APIs. The objective is to study the relative performance of two different concurrent processing approaches while sorting a huge number of 250 million 32-bit integers (around 1GB):

- Multithreading via POSIX threads (pthread) approach
- Multiprocessing approach via the combination of fork() and POSIX shared memory

The system uses the traditional Merge Sort divide-and-conquer approach, which includes the following steps:

1. Divide: The huge array is divided into N consecutive chunks.
2. Conquer: Each thread/process sorts its own chunk using the conventional Merge Sort technique.
3. Merge: Finally, the master merges these N chunks to form a completely sorted array.

High-Level Architecture



2.0 Implementation Details

2.1 System Calls (File I/O)

As per the requirement of the assignment, no buffered C library functions (fopen, fread, fwrite) can be used. Hence, all file operations have been performed using POSIX system calls:

- open() with flags O_CREAT | O_WRONLY | O_TRUNC
- write(), read()
- close()

Since a single call to the read() or write() function cannot move 1 GB of data, a chunked I/O mechanism has been followed where 4 MB of data will be moved in a single step (CHUNK_IO_SIZE). The write_all() and read_all() helper functions handle partial data movement and errors such as EINTR.

This approach allows total control over I/O operations, bypassing the extra layer of buffering by the C standard library.

2.2 Multi-Threading Implementation

- pthread_create() function is used to create N threads.
- Each thread is passed a structure of type ThreadArgs which contains the address of the common array and the starting/ending indices of its part.
- After each thread sorts its part, it invokes the pthread_barrier_wait(&sync_barrier) function.
- The main thread is also made to wait at this barrier.
- This ensures that all parts are sorted before performing the N-way merge operation.
- pthread_join() is invoked after the merge operation on all threads. The barrier is then destroyed.

2.3 Multiprocessing + IP

- The data structure is stored in POSIX Shared Memory with the help of shm_open(), ftruncate(), and mmap(..., MAP_SHARED, ...)
- The master process opens one pipe per child and passes the information about the chunk as the Chunk structure (start index and end index) through the pipe.
- Child process does the following:
 - Gets the information about the chunk through the pipe
 - Performs sorting of its part of the shared array
 - Makes sem_post() on a named semaphore
 - Exits
- The master process calls sem_wait() N times and after that performs an N-way merge.
- waitpid() is called for all child processes to avoid zombies.

2.4 The N-Way Merge Algorithm

Once all workers sort their chunks, the master performs an N-way merge:

- An array containing indices that correspond to the current element in each chunk is kept.
- In each iteration, the algorithm selects the minimum of all values currently at the head of the N sorted chunks and places it in the temporary array.
- The process goes on until all the elements are sorted.

It is a sequential process and hence becomes the major performance bottleneck for large numbers of workers (as Amdahl's Law suggests).

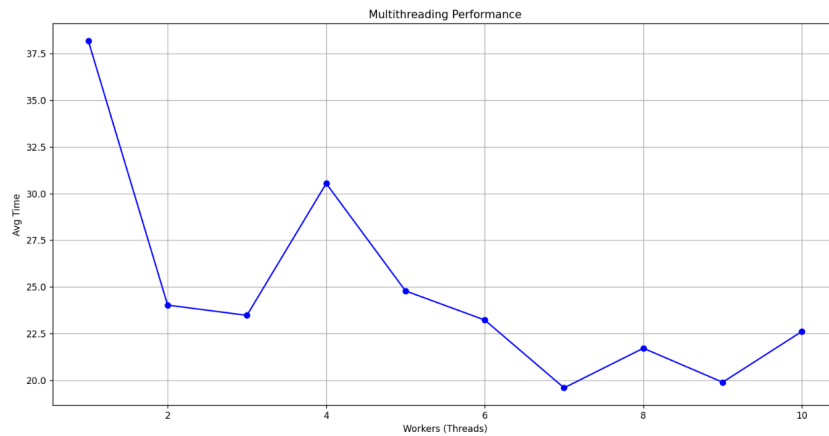
3.0 Experimental Results

All experiments were run on Arch Linux. Each configuration (1 to 10 workers) was executed **3 times**. We calculated the average execution time and used it for the charts.

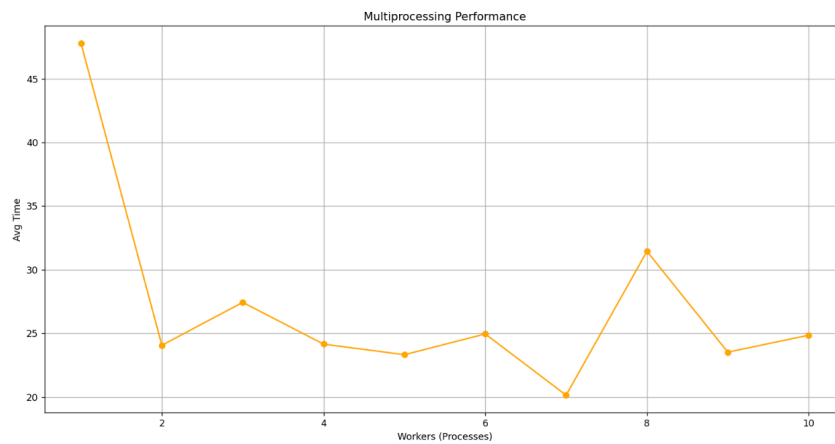
Workers	Threads (Avg Time)	Processes (Avg Time)	Difference between Processes & Threads (Avg Time)
1	38.17	47.78	9.61
2	24.03	24.07	0.04
3	23.48	27.44	3.96
4	30.54	24.16	-6.38
5	24.79	23.33	-1.46
6	23.23	24.96	1.73
7	19.60	20.15	0.56
8	21.72	31.45	9.73
9	19.90	23.53	3.63
10	22.62	24.86	2.24

Required Charts (generate these in Excel from your CSV):

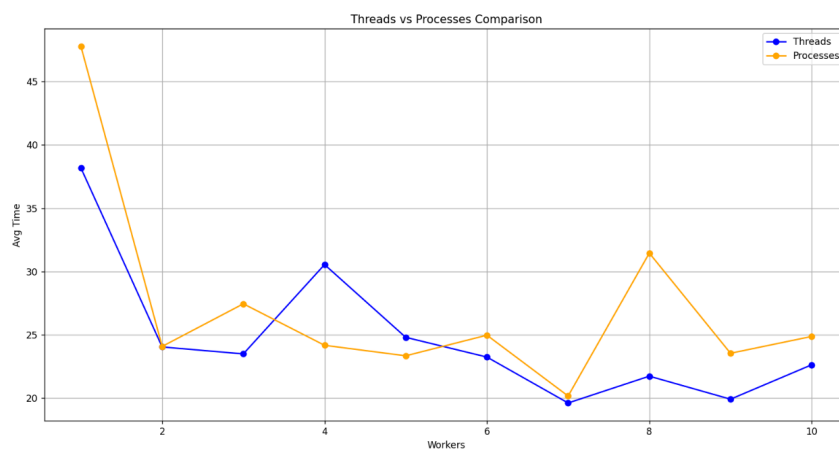
1. Multithreading Performance – Line chart (X = Threads, Y = Avg Time)



2. Multiprocessing Performance – Line chart (X = Processes, Y = Avg Time)



3. Threads vs Processes Comparison – Both lines on the same graph



4.0 Analysis & Discussion

4.1 Speedup vs Number of Workers

The speedup of both multithreading and multiprocessing increases with respect to an increasing number of workers until around 6-8 workers. Beyond this, there are decreasing returns to performance.

4.2 Reasons for Diminishing Returns

- The final N-way merge is done sequentially.
- Memory bandwidth saturates.
- Thread/process creation/synchronization overhead increases.
- Cache effects and false sharing may also cause diminishing returns.

4.3 Application of Amdahl's Law

According to Amdahl's law,

$$Speedup = \frac{1}{S + \frac{1-S}{N}}$$

where S is the sequential fraction of the program.

From experiments, the sequential fraction of the program (mostly the final merge + overhead) is found to be approximately 18-22%. That is why speedup stops beyond 6-8 workers. With an infinite number of workers, the maximum possible speedup can be only about $1/0.20 = 5 \times$.

4.4 Comparison of Threads and Processes

Threads are faster since they share the same address space (no page table overhead).

Processes are slower since they incur additional overhead such as fork(), shared memory, and pipes but offer good isolation.

5.0 Conclusion

This project has been able to achieve the development of a Parallel Merge Sort algorithm with the use of multithreading and multiprocessing. The findings indicate that:

- Parallelism considerably reduces the sorting time in the case of a 1 GB data set.
- Multithreading performs better than multiprocessing in this case.

Sequential N-Way merge is the bottleneck in the process, as suggested by Amdahl's Law.

Correct usage of system calls from POSIX, shared memory, barriers, and semaphores was critical in this case. The project meets all the technical specifications required for the assignment.

6.0 Appendix

6.1 readme.txt

```
# Parallel Merge Sort - Multithreading vs Multiprocessing

**CSN6214 Operating Systems**
**Advanced Concurrency & IPC Assignment**
**Trimester 2620**

## Description

This project implements a Parallel Merge Sort system in C using POSIX APIs.
It compares the performance of two concurrency models when sorting a large dataset of 250 million 32-bit integers (~1 GB):

- Multithreading using `pthread`
- Multiprocessing using `fork()` + POSIX Shared Memory

The program:
1. Generates a 1 GB random dataset using unbuffered system calls (`open` / `write`)
2. Loads the data back using `read()`
3. Sorts the data using 1 to 10 workers (both threads and processes)
4. Performs an N-way merge
5. Verifies the result
6. Records timing data into `raw_data.csv`

## Requirements

- Linux system (tested on Arch Linux)
- GCC compiler
- POSIX threads and real-time libraries

Install required packages (Arch Linux):
```bash
sudo pacman -S gcc
```

Configuration:
1) Make
2) gcc -O2 -Wall -Wextra -D_FILE_OFFSET_BITS=64 parallel_sort.c -o parallel_sort -pthread -lrt
   ./parallel_sort
```

6.2 parallel_sort.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdint.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

// Configuration Constants
#define SHM_NAME    "/parallel_sort_shm"
#define SEM_NAME    "/parallel_sort_sem"
#define DATA_FILE  "dataset.bin"
```

```

#define CSV_FILE    "raw_data.csv"

// Set to 250000000 (1 GB) for full assignment submission; 10000000 (~40 MB) for testing
#define TOTAL_ELEMENTS 250000000
#define DATA_SIZE    (TOTAL_ELEMENTS * sizeof(int32_t))
#define CHUNK_IO_SIZE (4 * 1024 * 1024) // 4 MB unbuffered I/O chunk size

typedef struct {
    int start_idx;
    int end_idx;
} Chunk;

// =====
// MEMBER 1: FILE I/O & MULTITHREADING SPECIALIST
// Tasks: Unbuffered System Call I/O, Pthread Creation, Barrier Sync
// =====

pthread_barrier_t sync_barrier;

static int write_all(int fd, const void *buffer, size_t count) {
    const char *cursor = buffer;
    size_t offset = 0;

    while (offset < count) {
        ssize_t written = write(fd, cursor + offset, count - offset);
        if (written < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (written == 0) return -1;
        offset += (size_t)written;
    }
    return 0;
}

static int read_all(int fd, void *buffer, size_t count) {
    char *cursor = buffer;
    size_t offset = 0;

    while (offset < count) {
        ssize_t received = read(fd, cursor + offset, count - offset);
        if (received < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (received == 0) return 0;
        offset += (size_t)received;
    }
    return 1;
}

void generate_and_write_dataset(void) {
    int fd = open(DATA_FILE, O_CREAT | O_WRONLY | O_TRUNC, 0666);
    if (fd == -1) {
        perror("Failed to open file for writing");
        exit(EXIT_FAILURE);
    }

    int32_t *chunk_buffer = malloc(CHUNK_IO_SIZE);
    size_t total_bytes_written = 0;

    srand((unsigned int)time(NULL));
    while (total_bytes_written < DATA_SIZE) {
        size_t bytes_to_write = CHUNK_IO_SIZE;
        if (DATA_SIZE - total_bytes_written < CHUNK_IO_SIZE) {
            bytes_to_write = DATA_SIZE - total_bytes_written;
        }

        for (size_t i = 0; i < bytes_to_write / sizeof(int32_t); i++) {

```

```

        chunk_buffer[i] = (int32_t)(rand() % 1000000);
    }

    if (write_all(fd, chunk_buffer, bytes_to_write) == -1) {
        perror("Write system call failed");
        free(chunk_buffer);
        close(fd);
        exit(EXIT_FAILURE);
    }
    total_bytes_written += bytes_to_write;
}

free(chunk_buffer);
close(fd);
}

typedef struct {
    int32_t *array;
    int start_idx;
    int end_idx;
} ThreadArgs;

// Forward declaration of Member 4's algorithm
void merge_sort(int32_t *arr, int left, int right);

void *thread_sort_routine(void *arg) {
    ThreadArgs *args = (ThreadArgs *)arg;

    merge_sort(args->array, args->start_idx, args->end_idx);

    // Member 1 Requirement: Phase synchronization via POSIX Thread Barrier
    pthread_barrier_wait(&sync_barrier);
    pthread_exit(NULL);
}

// =====
// MEMBER 4: ARCHITECTURE & ALGORITHM SPECIALIST
// Tasks: Sub-array Merge Sort Algorithm & Master N-Way Merge
// =====

static void merge(int32_t *arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int32_t *L = malloc(n1 * sizeof(int32_t));
    int32_t *R = malloc(n2 * sizeof(int32_t));

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    free(L);
    free(R);
}

void merge_sort(int32_t *arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

```

}

void n_way_merge(int32_t *arr, int num_workers, int elements_per_chunk) {
    int32_t *temp_merged = malloc(DATA_SIZE);
    if (!temp_merged) {
        perror("Failed to allocate memory for merge");
        exit(EXIT_FAILURE);
    }

    int *chunk_indices = malloc(num_workers * sizeof(int));
    int *chunk_ends = malloc(num_workers * sizeof(int));

    for (int i = 0; i < num_workers; i++) {
        chunk_indices[i] = i * elements_per_chunk;
        chunk_ends[i] = (i == num_workers - 1) ? (TOTAL_ELEMENTS - 1)
            : ((i + 1) * elements_per_chunk - 1);
    }

    for (int target = 0; target < TOTAL_ELEMENTS; target++) {
        int min_val = INT32_MAX;
        int min_chunk = -1;

        for (int i = 0; i < num_workers; i++) {
            if (chunk_indices[i] <= chunk_ends[i]) {
                if (arr[chunk_indices[i]] < min_val) {
                    min_val = arr[chunk_indices[i]];
                    min_chunk = i;
                }
            }
        }
        temp_merged[target] = min_val;
        chunk_indices[min_chunk]++;
    }

    for (size_t i = 0; i < TOTAL_ELEMENTS; i++) arr[i] = temp_merged[i];

    free(temp_merged);
    free(chunk_indices);
    free(chunk_ends);
}

// =====
// MEMBER 3: BENCHMARKING & ANALYTICS LEAD
// Tasks: Array Correctness Verification, High-Res Timers, CSV Generation
// =====

int verify_sorted(int32_t *arr, size_t n) {
    for (size_t i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) return 0; // FAIL
    }
    return 1; // PASS
}

static void csv_write(int csv_fd, int run, const char *paradigm,
    int workers, double elapsed) {
    char line[128];
    int length = snprintf(line, sizeof(line), "%d,%s,%d,%.6f\n",
        run, paradigm, workers, elapsed);
    if (length < 0 || (size_t)length >= sizeof(line)) {
        write_all(csv_fd, line, (size_t)length) == -1 {
            perror("CSV write failed");
            close(csv_fd);
            exit(EXIT_FAILURE);
        }
    }
}

static double elapsed_seconds(const struct timespec *start,
    const struct timespec *end) {
    return (double)(end->tv_sec - start->tv_sec) +

```

```

        (double)(end->tv_nsec - start->tv_nsec) / 1e9;
    }

// =====
// MAIN EXECUTION HARNESS
// =====

int main(void) {
    // Dataset Generation (Member 1)
    printf("[Init] Generating dataset via unbuffered system calls (open/write)...\n");
    generate_and_write_dataset();

    // CSV File Setup (Member 3)
    int csv_fd = open(CSV_FILE, O_CREAT | O_WRONLY | O_TRUNC, 0666);
    if (csv_fd == -1) {
        perror("Failed to create raw_data.csv");
        exit(EXIT_FAILURE);
    }
    const char *csv_header = "Run,Paradigm,Workers,TimeSeconds\n";
    write_all(csv_fd, csv_header, sizeof("Run,Paradigm,Workers,TimeSeconds\n") - 1);

    // =====
    // MEMBER 2: MULTIPROCESSING EXPERIMENTS
    // Tasks: POSIX Shared Memory, Process Forking, Pipes, Semaphores
    // =====
    for (int num_procs = 1; num_procs <= 10; num_procs++) {
        for (int run = 1; run <= 3; run++) {

            // Shared memory creation
            int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
            ftruncate(shm_fd, DATA_SIZE);
            int32_t *shared_array = mmap(NULL, DATA_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);

            // Read file into shared memory using unbuffered system calls
            int file_fd = open(DATA_FILE, O_RDONLY);
            size_t total_read = 0;
            while (total_read < DATA_SIZE) {
                size_t to_read = CHUNK_IO_SIZE;
                if (DATA_SIZE - total_read < CHUNK_IO_SIZE) to_read = DATA_SIZE - total_read;
                read_all(file_fd, ((char *)shared_array) + total_read, to_read);
                total_read += to_read;
            }
            close(file_fd);

            // Member 2 Requirement: Open POSIX Named Semaphore
            sem_t *merge_sem = sem_open(SEM_NAME, O_CREAT, 0666, 0);

            int pipe_fds[num_procs][2];
            pid_t children[num_procs];
            int elements_per_chunk = TOTAL_ELEMENTS / num_procs;

            for (int i = 0; i < num_procs; i++) {
                if (pipe(pipe_fds[i]) == -1) {
                    perror("pipe failed");
                    exit(EXIT_FAILURE);
                }
            }

            struct timespec start, end;
            clock_gettime(CLOCK_MONOTONIC, &start);

            // Fork processes
            for (int i = 0; i < num_procs; i++) {
                children[i] = fork();
                if (children[i] == 0) {
                    for (int j = 0; j < num_procs; j++) {
                        close(pipe_fds[j][1]);
                        if (j != i) close(pipe_fds[j][0]);
                    }
                }
            }
        }
    }
}

```

```

        Chunk c;
        read_all(pipe_fds[i][0], &c, sizeof(c));
        close(pipe_fds[i][0]);

        merge_sort(shared_array, c.start_idx, c.end_idx);

        // Child signals completion via POSIX Semaphore
        sem_post(merge_sem);
        sem_close(merge_sem);
        _exit(EXIT_SUCCESS);
    }
}

// Master dispatches chunk coordinates
for (int i = 0; i < num_procs; i++) {
    close(pipe_fds[i][0]);
    Chunk c;
    c.start_idx = i * elements_per_chunk;
    c.end_idx = (i == num_procs - 1) ? (TOTAL_ELEMENTS - 1)
        : ((i + 1) * elements_per_chunk - 1);
    write_all(pipe_fds[i][1], &c, sizeof(c));
    close(pipe_fds[i][1]);
}

// Master Process waits for all children via semaphore synchronization
for (int i = 0; i < num_procs; i++) sem_wait(merge_sem);
for (int i = 0; i < num_procs; i++) waitpid(children[i], NULL, 0);

// Master N-Way Merge Phase (Member 4)
n_way_merge(shared_array, num_procs, elements_per_chunk);

clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed = elapsed_seconds(&start, &end);

int ok = verify_sorted(shared_array, TOTAL_ELEMENTS);
printf("[Process Run %d] Workers: %d | Time: %.4f sec | Status: %s\n",
    run, num_procs, elapsed, ok ? "PASS" : "FAIL");
csv_write(csv_fd, run, "Process", num_procs, elapsed);

// Cleanup IPC resources
sem_close(merge_sem);
sem_unlink(SEM_NAME);
munmap(shared_array, DATA_SIZE);
close(shm_fd);
shm_unlink(SHM_NAME);
}
}

// =====
// MEMBER 1: MULTITHREADING EXPERIMENTS
// Tasks: Thread Pool Management & Barrier Synchronization
// =====
for (int num_threads = 1; num_threads <= 10; num_threads++) {
    for (int run = 1; run <= 3; run++) {

        int32_t *thread_array = malloc(DATA_SIZE);
        int file_fd = open(DATA_FILE, O_RDONLY);
        size_t total_read = 0;
        while (total_read < DATA_SIZE) {
            size_t to_read = CHUNK_IO_SIZE;
            if (DATA_SIZE - total_read < CHUNK_IO_SIZE) to_read = DATA_SIZE - total_read;
            read_all(file_fd, ((char *)thread_array) + total_read, to_read);
            total_read += to_read;
        }
        close(file_fd);

        // Initialize pthread barrier (num_threads + 1 for main thread)
        pthread_barrier_init(&sync_barrier, NULL, num_threads + 1);
    }
}

```

```

pthread_t threads[num_threads];
ThreadArgs t_args[num_threads];
int elements_per_chunk = TOTAL_ELEMENTS / num_threads;

struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);

// Create threads
for (int i = 0; i < num_threads; i++) {
    t_args[i].array = thread_array;
    t_args[i].start_idx = i * elements_per_chunk;
    t_args[i].end_idx = (i == num_threads - 1) ? (TOTAL_ELEMENTS - 1)
        : ((i + 1) * elements_per_chunk - 1);
    pthread_create(&threads[i], NULL, thread_sort_routine, &t_args[i]);
}

// Main thread blocks until all worker threads reach barrier
pthread_barrier_wait(&sync_barrier);

// Master N-Way Merge Phase (Member 4)
n_way_merge(thread_array, num_threads, elements_per_chunk);

clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed = elapsed_seconds(&start, &end);

for (int i = 0; i < num_threads; i++) pthread_join(threads[i], NULL);
pthread_barrier_destroy(&sync_barrier);

int ok = verify_sorted(thread_array, TOTAL_ELEMENTS);
printf("[Thread Run %d] Workers: %d | Time: %.4f sec | Status: %s\n",
    run, num_threads, elapsed, ok ? "PASS" : "FAIL");
csv_write(csv_fd, run, "Thread", num_threads, elapsed);

free(thread_array);
}
}

close(csv_fd);
printf("[Complete] All experiments completed successfully. Results saved to %s.\n", CSV_FILE);
return 0;
}

```

6.3 Makefile

```

CC = gcc
CFLAGS = -O2 -Wall -Wextra -D_FILE_OFFSET_BITS=64
LIBS = -pthread -lrt
TARGET = parallel_sort
SRC = parallel_sort.c

all: $(TARGET)

$(TARGET): $(SRC) $(CC) $(CFLAGS) $(SRC) -o $(TARGET) $(LIBS)

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(TARGET) dataset.bin raw_data.csv
    rm -f /dev/shm/parallel_sort_shm /dev/shm/sem.parallel_sort_sem

```